# Chapitre 3

## Commande neuronale à minimum de variance

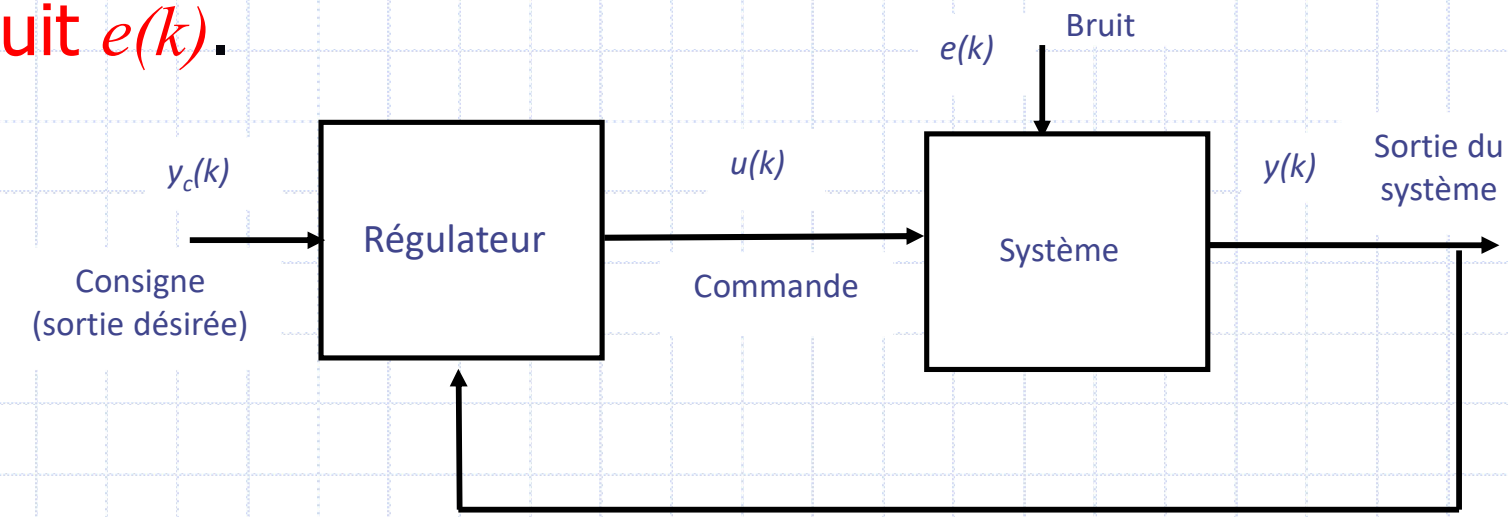
# Contenu

- Principe de la commande à minimum de variance
- Critère à minimiser
- Calcul du prédicteur à un pas
- Calcul de la commande
- Exemple de simulation

# Principe de la commande à minimum de variance:

## Problème:

Trouver la commande $u(k)$ permettant à la sortie $y(k)$ du système d'atteindre la consigne $yc(k)$ en minimisant les effets de bruit $e(k)$.




## Système non linéaire:

$$y(k) = h(y(k-1),\ldots,y(k-n)\ \ u(k-1),\ldots,u(k-m))$$

$h(.)$ : fonction non linéaire inconnue

## Exemple:

$$y(k) = 2y(k-1) + (u(k-1))^3 + \frac{1 + y(k-2)}{2 + u(k-1)y(k-1)}$$

$$y(k+1) = h(y(k),\ldots,y(k-n+1)\ \ u(k),\ldots,u(k-m+1))$$

A l'itération k:

$y(k), y(k-1),\ldots, y(k-n+1), u(k-1),\ldots, u(k-m+1)$: sont connus.

$u(k)$: inconnu.

Question: Déterminer $u(k)$ tel que: $y(k+1) = yc(k+1)$.

Critère à minimiser:

$$\min_{u(k)} J = \min_{u(k)} E\left(y_c(k+1) - y(k+1)\right)^2$$

$E$: espérance mathématique

Problème:

On ne peut pas évaluer $y(k+1)$ car $h(.)$ est inconnue.

Solution:

Prédiction de la valeur future de la sortie en utilisant le modèle du système.

$$\hat{y}(k+1) = ?$$

## Modèle neuronal:

$$\hat{y}(k) = NN(y(k-1),\ldots,y(k-n)\ \ u(k-1),\ldots,u(k-m))$$

Erreur de modélisation: $e(k+1) = y(k+1) - \hat{y}(k+1)$

$$\hat{y}(k+1) = NN(y(k),\ldots,y(k-n+1)\ \ u(k),\ldots,u(k-m+1))$$

$y(k), y(k-1), \ldots, y(k-n+1), u(k-1), \ldots, u(k-m+1)$: sont connus.

$u(k)$: inconnu.

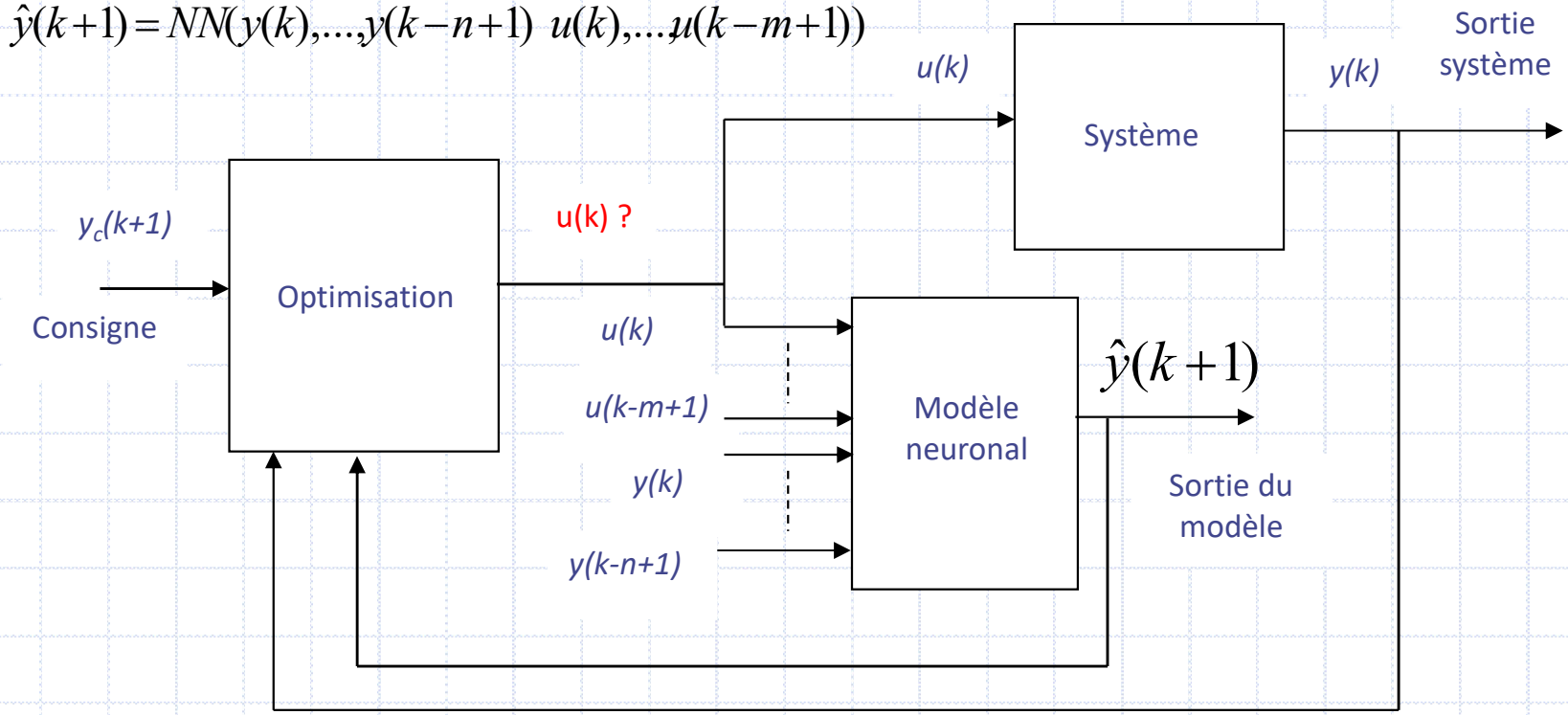
Question: Déterminer $u(k)$ tel que: $\hat{y}(k+1) = y_c(k+1)$

# Exploitation du modèle neuronal

On considère le modèle déterminé lors de la phase d'apprentissage.

## Sortie du modèle:

$$\hat{y}(k+1) = NN(y(k),\ldots,y(k-n+1)\ u(k),\ldots,u(k-m+1))$$




$$\min_{u(k)} J = \min_{u(k)} E\left(y_c(k+1) - y(k+1)\right)^2$$

Question: Déterminer $u(k)$ tel que: $\hat{y}(k+1) = y_c(k+1)$

Critère à minimiser:

$$J = E(y_c(k+1) - y(k+1))^2$$

$$e(k+1) = y(k+1) - \hat{y}(k+1)$$

$$J = E\left(y_c(k+1) - y(k+1)\right)^2 = E\left(y_c(k+1) - \hat{y}(k+1) - e(k+1)\right)^2$$

- La séquence de l'erreur est supposée indépendante des autres mesures.
- Le terme $E\left(e(k+1)\right)^2$ tend vers une valeur finie (la variance de l'erreur).

$$J = E\left(y_c(k+1) - \hat{y}(k+1)\right)^2 - 2E\left((y_c(k+1) - \hat{y}(k+1))e(k+1)\right) + E\left(e(k+1)\right)^2$$

$$E\left(e(k+1)\right) = 0 \;\; et \;\; E\left(e(k+1)\right)^2 = \delta_e^2$$

$$J = E\left(y_c(k+1) - \hat{y}(k+1)\right)^2 + \delta_e^2$$

$$J = E\left(y_c(k+1) - \hat{y}(k+1)\right)^2 - 2E\left((y_c(k+1) - \hat{y}(k+1))e(k+1)\right) + E\left(e(k+1)\right)^2$$

$$E\left(e(k+1)\right) = 0 \;\; et \;\; E\left(e(k+1)\right)^2 = \delta_e^2$$

$$J = E\left(y_c(k+1) - \hat{y}(k+1)\right)^2 + \delta_e^2$$

Le critère $J$ est minimal si: $E\left(y_c(k+1) - \hat{y}(k+1)\right)^2 = 0$

$$\min_{u(k)} J = \min_{u(k)} E\left(y_c(k+1) - y(k+1)\right)^2 = \min_{u(k)} \left(y_c(k+1) - \hat{y}(k+1)\right)^2$$

Solution:

$$u_i(k) = u_{i-1}(k) - \eta \frac{\partial J}{\partial u_{i-1}(k)} \qquad i = 1,\ldots,i_{\max}$$

$$\frac{\partial J}{\partial u_{i-1}(k)} = -2(y_c(k+1) - \hat{y}(k+1)) \frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$$

## Les étapes de l'algorithme de commande

$i=1$ : valeur initiale de $i$

$u_0(k)$ : valeur initiale de la commande

*Boucle de calcul de u(k)* *(for i=1:imax)*

- Calcul de la sortie prédite en utilisant le modèle

$$\hat{y}(k+1)=NN(y(k),...,y(k-n+1)\ \ u(k),...,u(k-m+1))$$

- Calcul de:

$$\frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$$

- Calcul de:

$$\frac{\partial J}{\partial u_{i-1}(k)}=-2(y_c(k+1)-\hat{y}(k+1))\frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$$

- Calcul de la commande:

$$u_i(k)=u_{i-1}(k)-\eta\frac{\partial J}{\partial u_{i-1}(k)}$$

- Incrémentation de $i$: $i=i+1$

Jusqu'à: $i=i_{\max}$ *(fin boucle i)*

# Algorithme de simulation

✓ *Introduction de:* $N \quad y_c(k), k=1,\ldots,N \quad i=1 \quad i_{\max} \quad u_0 \quad \eta$

✓ *Paramètres du modèle neuronal*

*Boucle de calcul* *(for k=1:N)*

- Sortie du système: $y(k)$

*Boucle de calcul de u(k)* *(for i=1:imax)*

- Calcul de la sortie prédite: $\hat{y}(k+1)$

- Calcul de: $\dfrac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$ et de: $\dfrac{\partial J}{\partial u_{i-1}(k)}$

- Calcul de la commande: $u_i(k) = u_{i-1}(k) - \eta \dfrac{\partial J}{\partial u_{i-1}(k)}$

- Incrémentation de *i:* $i = i+1$

*Jusqu'à:* $i = i_{\max}$ *(fin boucle i)*

- prendre: $u(k) = u_{i_{\max}}(k)$

*Fin calcul* *(fin boucle k)*

Affichage des résultats

# Algorithme de simulation

✓ *Introduction de:* $N \quad y_c(k), k=1,\ldots,N \quad i=1 \quad i_{\max} \quad u_0 \quad \eta$

✓ *Paramètres du modèle neuronal*

*Boucle de calcul* *(for k=1:N)*

- Sortie du système: $y(k)$

*Boucle de calcul de u(k)* *(for i=1:imax)*

- Calcul de la sortie prédite: $\hat{y}(k+1)$

- Calcul de: $\dfrac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$ et de: $\dfrac{\partial J}{\partial u_{i-1}(k)}$

- Calcul de la commande: $u_i(k) = u_{i-1}(k) - \eta \dfrac{\partial J}{\partial u_{i-1}(k)}$

- Incrémentation de *i:* $i = i+1$

*Jusqu'à:* $i = i_{\max}$ *(fin boucle i)*

- prendre: $u(k) = u_{i_{\max}}(k)$

*Fin calcul* *(fin boucle k)*

Affichage des résultats

*On introduit la variation de la commande avec une pondération dans le critère de performance:*

$$J = \left(y_c(k+1) - \hat{y}(k+1)\right)^2 + \lambda\left(u(k) - u(k-1)\right)^2$$

$$\min_{u(k)} J = \min_{u(k)} \left\{ \left(y_c(k+1) - \hat{y}(k+1)\right)^2 + \lambda\left(u(k) - u(k-1)\right)^2 \right\}$$

## Solution:

$$u_i(k) = u_{i-1}(k) - \eta \frac{\partial J}{\partial u_{i-1}(k)} \qquad i = 1,\ldots,i_{\max}$$

$$\frac{\partial J}{\partial u_{i-1}(k)} = -2(y_c(k+1) - \hat{y}(k+1)) \frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)} + 2\lambda(u(k) - u(k-1))$$

# Correction de l'erreur en régime permanent

- ✓ *Parfois, on trouve, en régime permanent, une erreur entre la consigne et la sortie du système.*

$$e_\infty = y_c - y_\infty \neq 0$$

- ✓ *Pour annuler cette erreur, on corrige les valeurs données par le prédicteur neuronal:*

$$\hat{y}(k+1) = \hat{y}(k+1) + \alpha(y(k) - \hat{y}(k))$$

- ✓ *Le paramètre* $\alpha$ *est utilisé pour obtenir une erreur nulle. On change la valeur de* $\alpha$ *jusqu'à l'obtention d'une erreur en régime permanent très proche de zéro.*

# Correction de l'erreur en régime permanent

*Boucle de calcul* *(for k=1:N)*

- Sortie du système: $y(k)$
- Sortie du modèle: $\hat{y}(k)$

*Boucle de calcul de u(k)* *(for i=1:imax)*

- Calcul de la sortie prédite: $\hat{y}(k+1)$
- Correction de la prédiction: $\hat{y}(k+1) = \hat{y}(k+1) + \alpha(y(k) - \hat{y}(k))$
- Calcul de: $\frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$ et de: $\frac{\partial J}{\partial u_{i-1}(k)}$
- Calcul de la commande: $u_i(k) = u_{i-1}(k) - \eta \frac{\partial J}{\partial u_{i-1}(k)}$
- Incrémentation de *i:* $i = i+1$

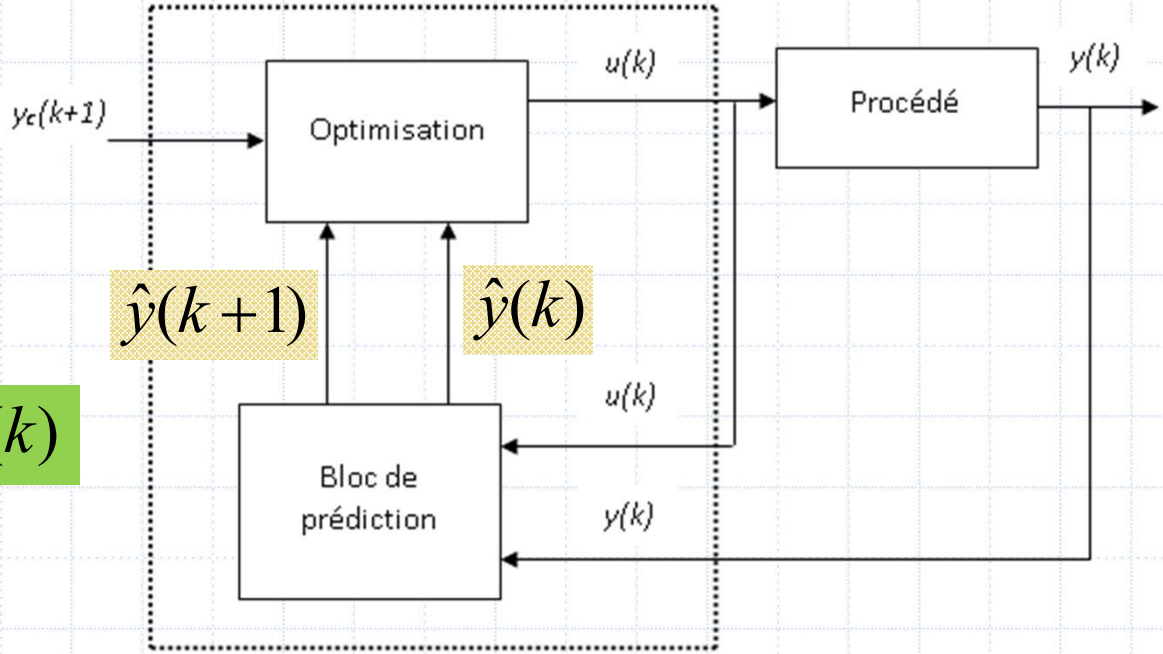
*Jusqu'à:* $i = i_{\max}$ *(fin boucle i)*

- prendre: $u(k) = u_{i_{\max}}(k)$

*Fin calcul* *(fin boucle k)*

Affichage des résultats

# Schéma bloc du système en boule fermée




## Bloc Optimisation

Calcul de la commande:

$$\frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)} \qquad \frac{\partial J}{\partial u_{i-1}(k)} \qquad u_i(k)$$

## Bloc de prédiction

Calcul des sorties prédites:

$$\hat{y}(k) \qquad \hat{y}(k+1)$$

# Exemple

- Système

$$y(k) = -0.8\frac{(u(k-1))^2}{1+(u(k-1))^2}y(k-2) + 0.033y(k-1) + 0.5u(k-1)$$

- Modèle (un seul neurone)

$$v = -w_0 + w_1 y(k-1) + w_2 u(k-1)$$
$$= 5\ 10^{-3} + 0{,}032y(k-1) + 0{,}508u(k-1)$$

$$\hat{y}(k) = f_2(v)$$

## Prédicteur à un pas:

$$v(k+1) = -w_0 + w_1 y(k) + w_2 u(k)$$

$$\hat{y}(k+1) = f_2(v(k+1))$$

## Critère à minimiser

$$J = \tfrac{1}{2}(y_c(k+1) - \hat{y}(k+1))^2$$

## Calcul de la dérivée

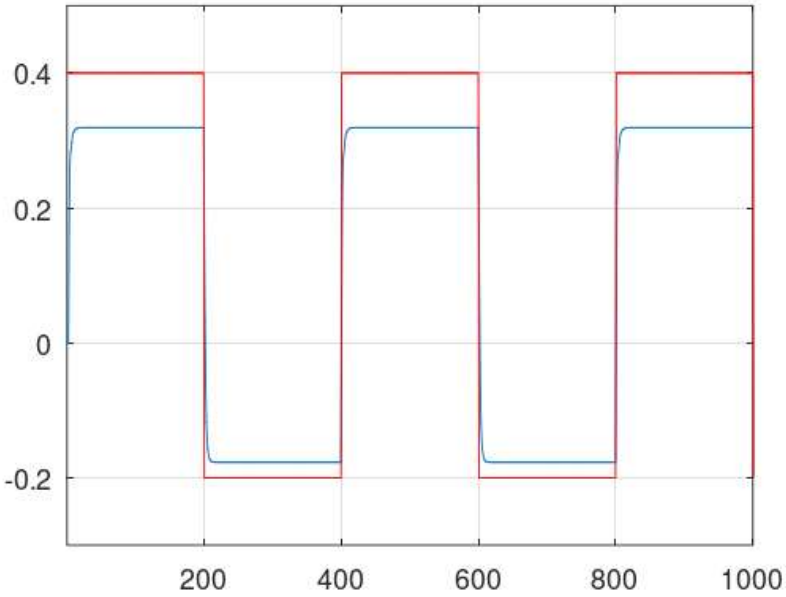
$$\frac{\partial J}{\partial u_{i-1}(k)} = -(y_c(k+1) - \hat{y}(k+1)) \frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$$

$$\frac{d\hat{y}(k+1)}{du(k)} = \frac{d\hat{y}(k+1)}{dv(k+1)} \frac{dv(k+1)}{du(k)}$$

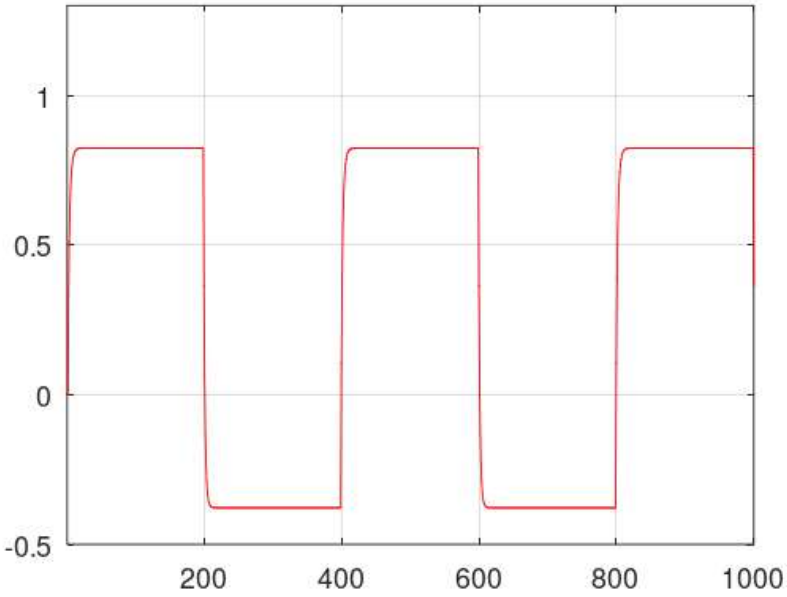
$$\frac{d\hat{y}(k+1)}{du(k)} = w_2 f_2'(v(k+1)$$

# Résultats

$i_{\max} = 10$ $\eta = 0{,}2$ $\alpha = 0$




Evolution de la consigne et de la sortie du système.




Evolution de la commande.

- Présence d'une erreur en régime permanent.

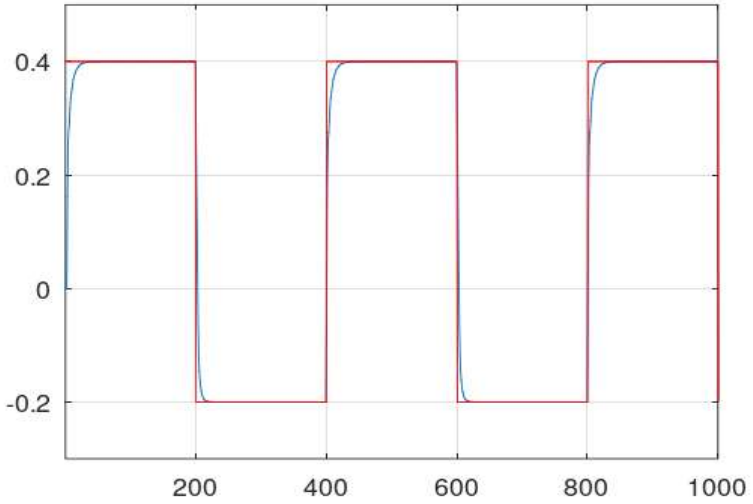
$$e_\infty = y_c - y_\infty \neq 0$$

- On introduit le terme de correction du prédicteur.

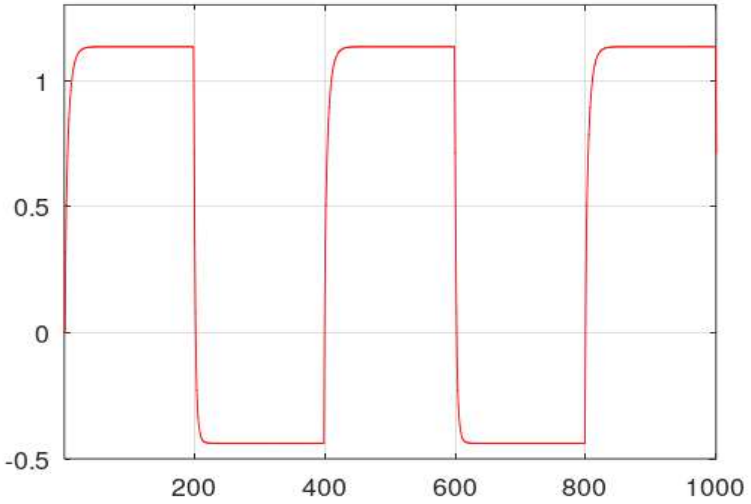
$$\hat{y}(k+1) = \hat{y}(k+1) + \alpha(y(k) - \hat{y}(k))$$

# Résultats

$$i_{\max} = 10 \quad \eta = 0{,}2 \quad \alpha = 1$$




Evolution de la consigne et de la sortie du système.



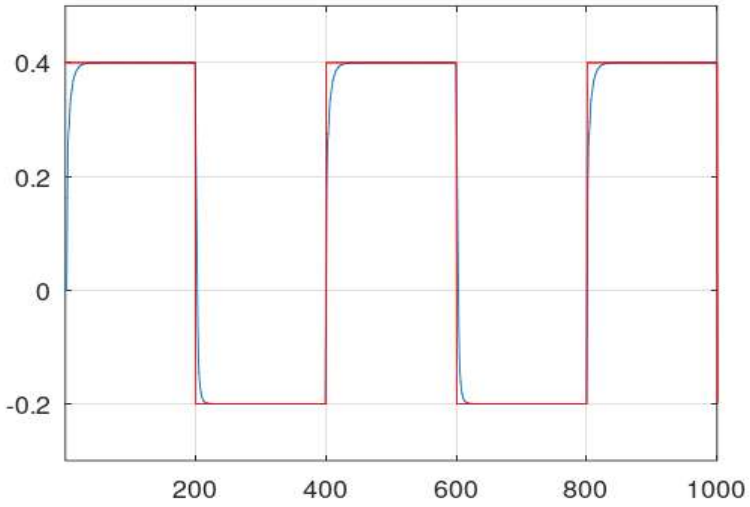

Evolution de la commande.

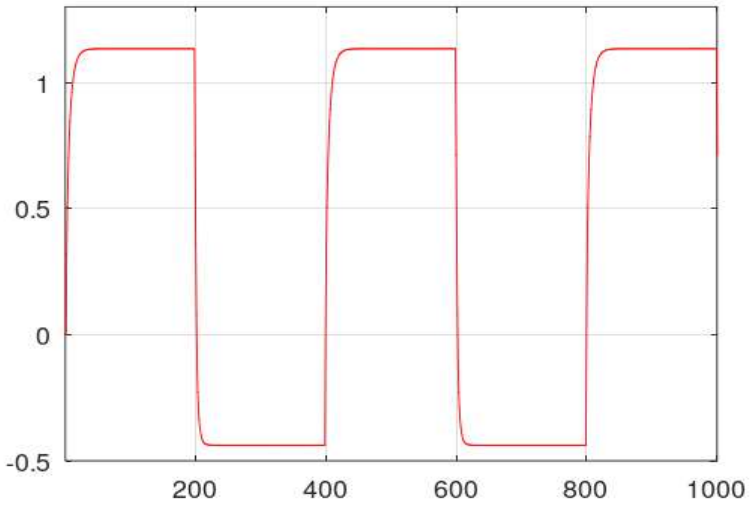
Avec la correction du prédicteur, on obtient une erreur en régime permanent nulle.

$$\hat{y}(k+1) = \hat{y}(k+1) + \alpha(y(k) - \hat{y}(k))$$

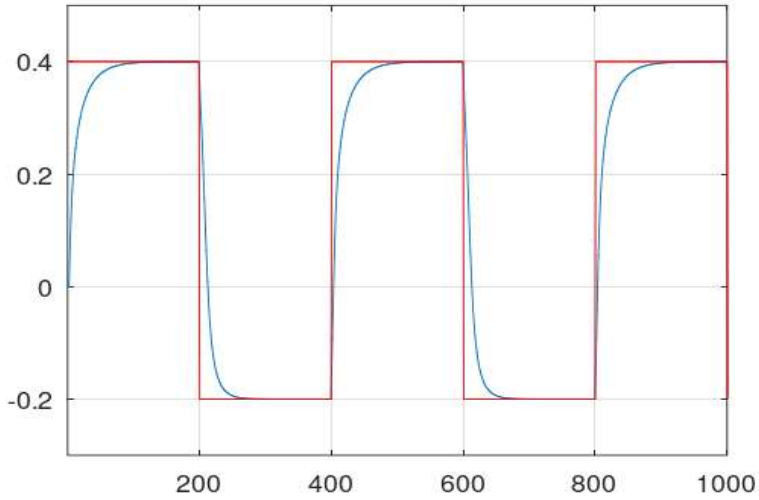
# Résultats

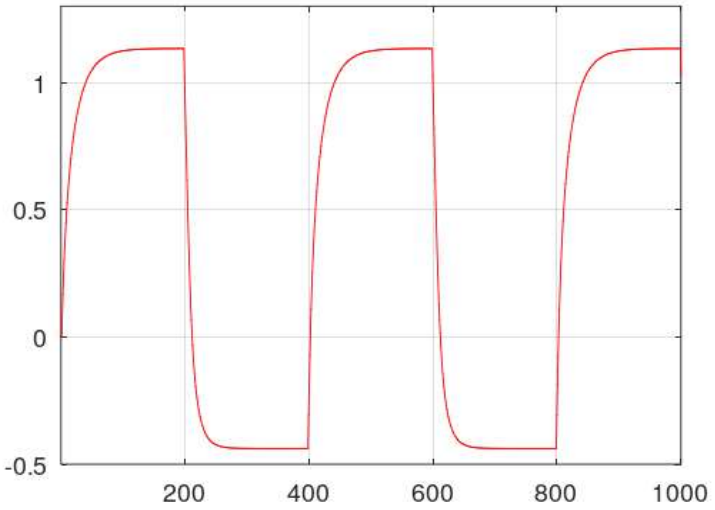
$i_{\max} = 10 \quad \eta = 0{,}2 \quad \alpha = 1 \quad \lambda = 0$




$i_{\max} = 10 \quad \eta = 0{,}2 \quad \alpha = 1 \quad \lambda = 1$



## Exemple: système non linéaire

$$y(k) = -0.8\frac{(u(k-1))^2}{1+(u(k-1))^2}y(k-2) + 0.033y(k-1) + 0.5u(k-1)$$

## Prédicteur à un pas:

$$\hat{y}(k+1) = f_2(v(k+1)) \qquad v(k+1) = -w_0 + w_1 s_1(k+1) + w_2 s_2(k+1)$$

$$s_1(k+1) = f_1(v_1(k+1)) \qquad v_1(k+1) = -w_{10} + w_{11} y(k) + w_{12} u(k)$$

$$s_2(k+1) = f_1(v_2(k+1)) \qquad v_2(k+1) = -w_{20} + w_{21} y(k) + w_{22} u(k)$$

$$\frac{\partial J}{\partial u_{i-1}(k)} = -(y_c(k+1) - \hat{y}(k+1))\frac{\partial \hat{y}(k+1)}{\partial u_{i-1}(k)}$$

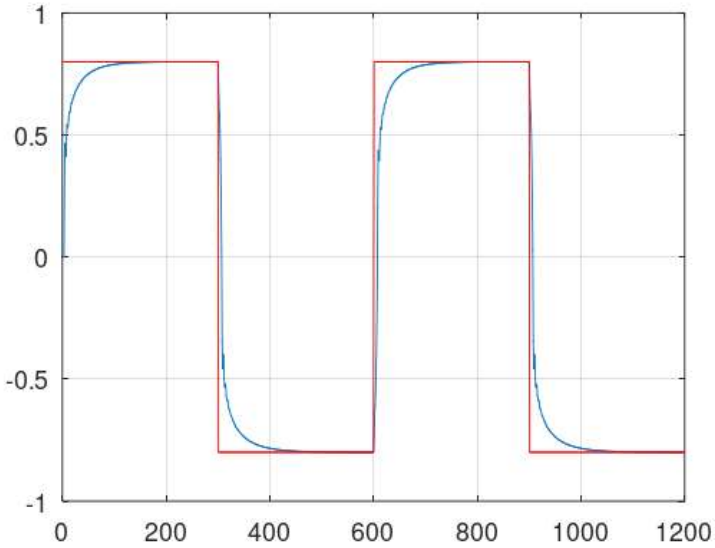
## Calcul de la dérivée

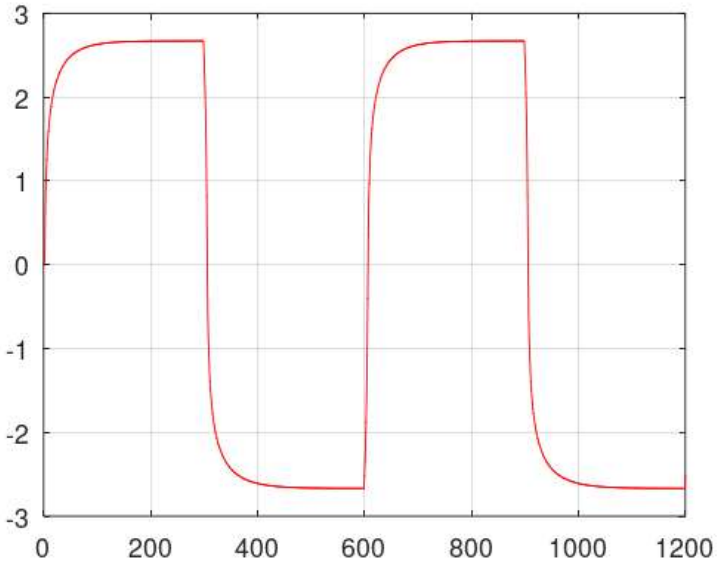
$$\frac{d\hat{y}(k+1)}{du(k)} = \frac{d\hat{y}(k+1)}{dv(k+1)}\left(\frac{dv(k+1)}{ds_1(k+1)}\frac{ds_1(k+1)}{du(k)} + \frac{dv(k+1)}{ds_2(k+1)}\frac{ds_2(k+1)}{du(k)}\right)$$

$$\frac{d\hat{y}(k+1)}{du(k)} = f_2'(v(k+1)\left(w_1 f_1'(v_1(k+1))w_{12} + w_2 f_1'(v_2(k+1))w_{22}\right)$$

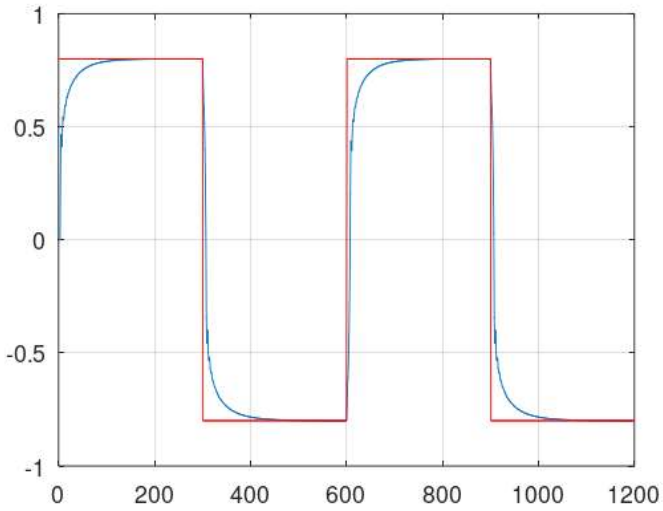
# Résultats

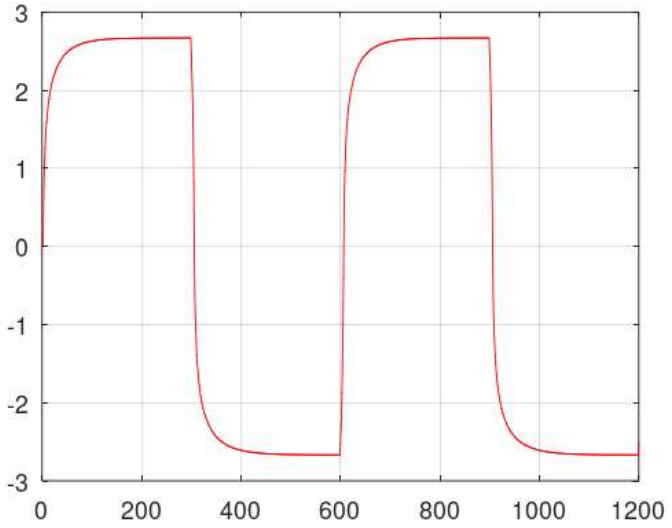
$i_{\max} = 10 \quad \eta = 0{,}2 \quad \alpha = 1 \quad \lambda = 0$



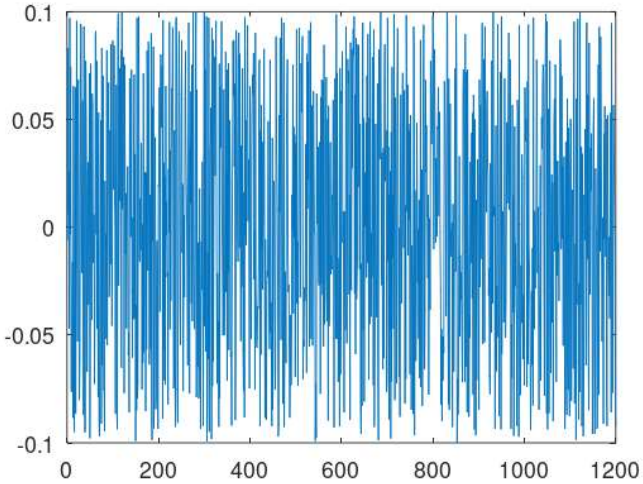

$i_{\max} = 10 \quad \eta = 0{,}2 \quad \alpha = 1 \quad \lambda = 1$



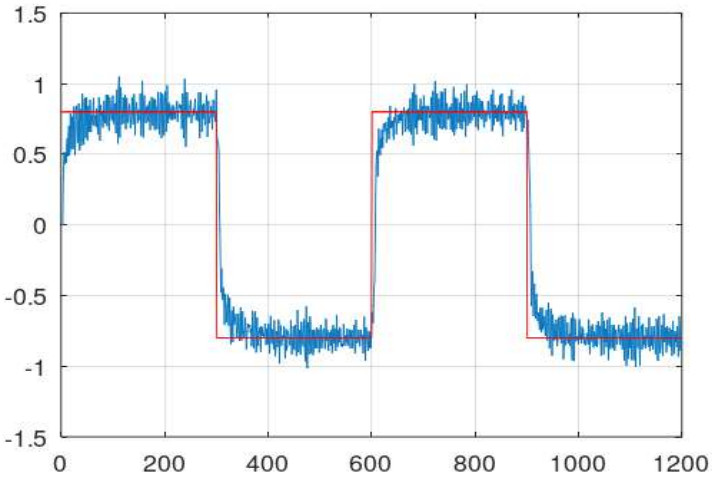



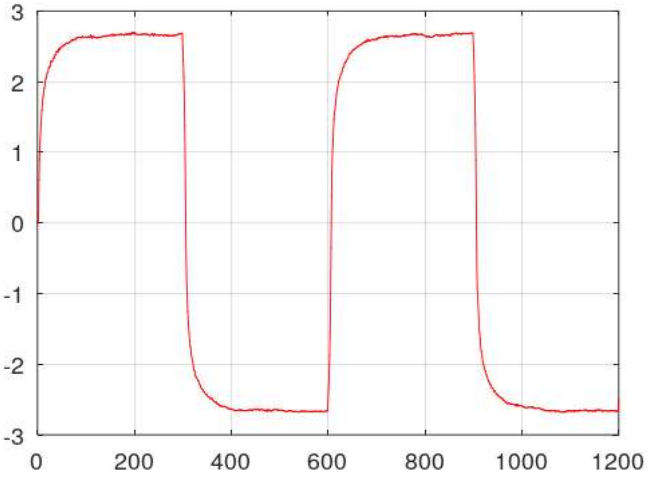
# Résultats

$i_{\max} = 10 \quad \eta = 0{,}2 \quad \alpha = 1 \quad \lambda = 0$




Bruit ajouté à la sortie du système



# Exemple

- Système

$$s(k) = 0.5y(k-1) + 0.5y(k-2)$$

$$y(k) = 0.2y(k-2) + 1.2u(k-1) + 0.2(y(k-1))^2 + 0.4\sin(s(k))\cos(s(k))$$

- Modèle neuronal:

  - 10 neurones dans la couche cachée, fonction tangente sigmoïde.
  - 1 neurone dans la couche de sortie, fonction linéaire
  - entrée: y(k-1), y(k-2) et u(k-1)

- Sorties des neurones de la couche cachée

$$s_j(k) = f\left(w1_{j1}\, y(k-1) + w1_{j2}\, y(k-2) + w1_{j3}\, u(k-1) - w_{j4}\right)$$

$$j = 1, \ldots, 10 \qquad f(v) = \frac{1 - e^{-2\beta v}}{1 + e^{-2\beta v}}$$

- Sortie du réseau

$$\hat{y}(k) = -w2_{11} + \sum_{i=1}^{10} w2_i\, s_i(k)$$

# Exemple

- Expression du prédicteur: $\hat{y}(k+1)$
- Sorties des neurones de la couche cachée

$$s_j(k+1) = f\left(w1_{j1}\, y(k) + w1_{j2}\, y(k-1) + w1_{j3}\, u(k) - w_{j4}\right)$$

$$j = 1, \ldots, 10$$

$$f(v) = \frac{1 - e^{-2\beta v}}{1 + e^{-2\beta v}}$$

- Sortie du réseau

$$\hat{y}(k+1) = -w2_{11} + \sum_{i=1}^{10} w2_i\, s_i(k+1)$$

- Calcul de la dérivée

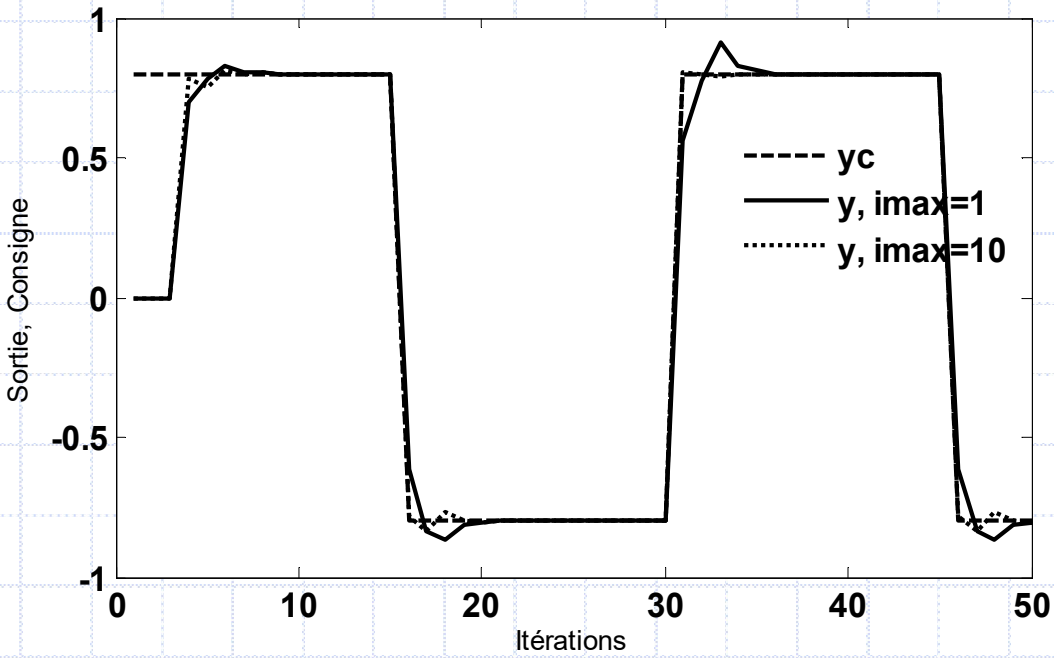
$$\frac{d\hat{y}(k+1)}{du(k)} = \sum_{i=1}^{10} w2_i \frac{ds_i(k+1)}{du(k)}$$

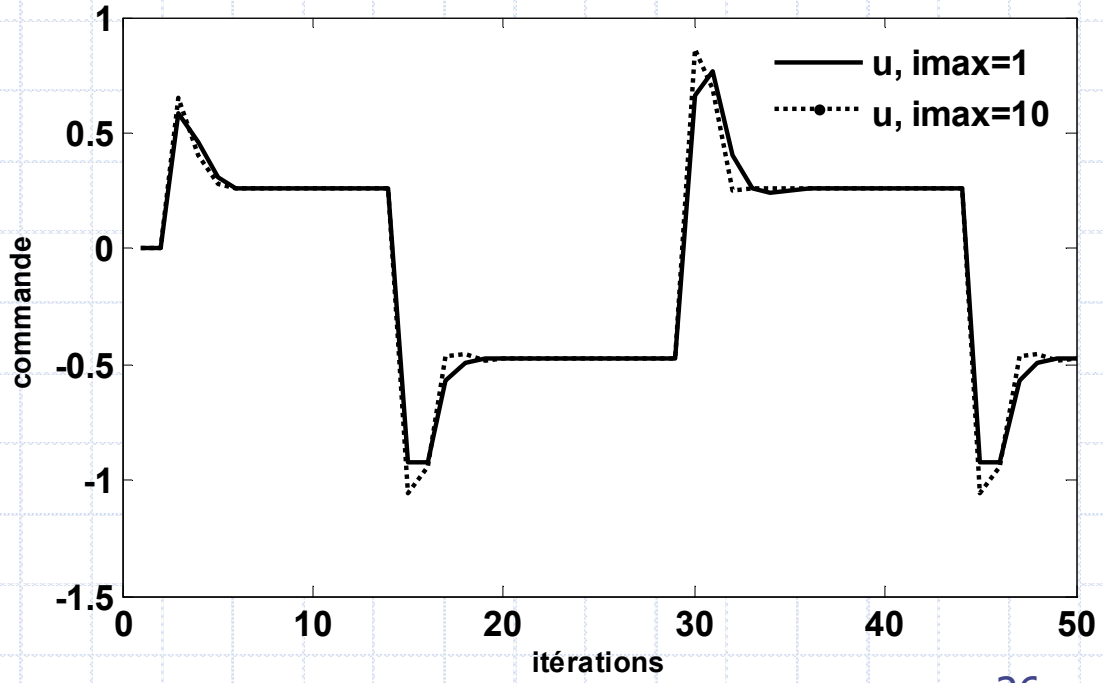
$$\frac{ds_j(k+1)}{du(k)} = w1_{j3}\, f'\left(w1_{j1}\, y(k) + w1_{j2}\, y(k-1) + w1_{j3}\, u(k) - w_{j4}\right)$$

# Résultats

$i_{\max} = 1 \quad \eta = 0{,}3$

$i_{\max} = 10 \quad \eta = 0{,}3$

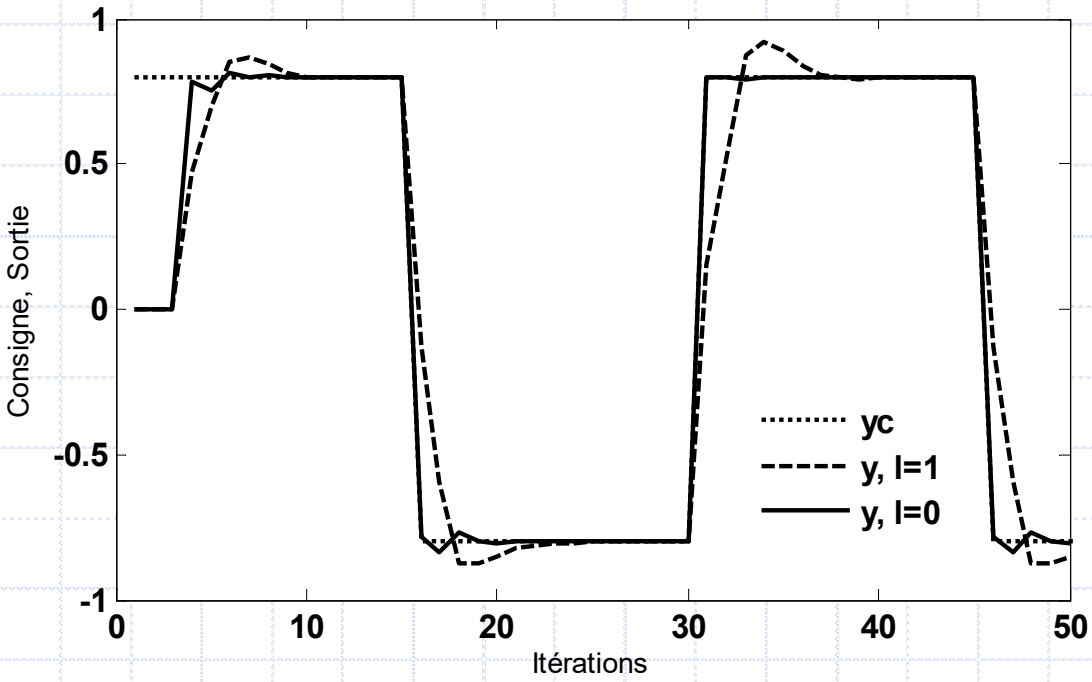


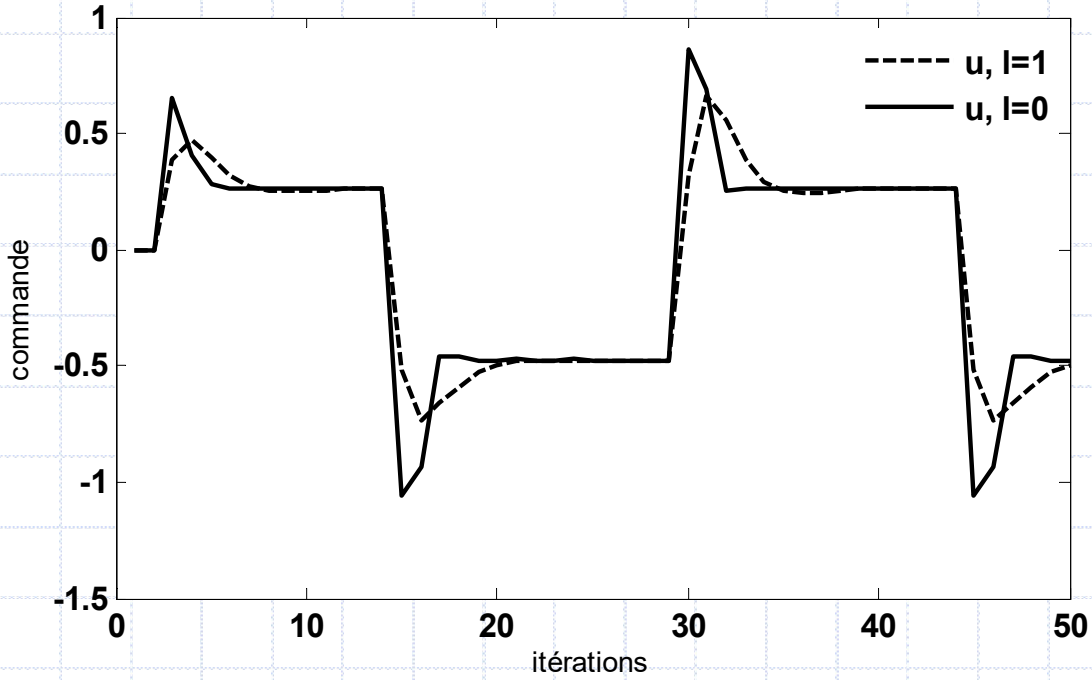



# Résultats

$i_{\max} = 10 \quad \eta = 0{,}3$

$i_{\max} = 10 \quad \eta = 0{,}3$



FIN